

Contents

Chapitre 1 — État de l’Art : LLMs Embarqués sur Smartphone	1
1. Introduction et contexte	1
2. Panorama des modèles disponibles pour smartphone	2
2.1 Chronologie de l’émergence des LLMs mobiles	2
2.2 Familles de modèles et acteurs principaux	2
3. Comparaison des modèles : taille, latence, capacités	4
3.1 Tableau de comparaison général	4
3.2 Contraintes matérielles	5
4. Frameworks d’inférence mobile	6
4.1 llama.cpp	6
4.2 ML Kit GenAI (AICore)	6
4.3 LiteRT (anciennement TensorFlow Lite)	6
4.4 MLC-LLM	6
4.5 MediaPipe LLM Inference	7
4.6 Comparaison des frameworks	7
5. Exécution locale pure vs architecture hybride (edge + cloud)	7
5.1 Exécution 100% locale	7
5.2 Architecture hybride (edge + cloud)	8
5.3 Positionnement des solutions étudiées	8
6. Critères de choix d’une solution	9
7. Tendances et perspectives (2025–2027)	9
7.1 Compression agressive des modèles	9
7.2 NPU comme cible principale	9
7.3 LLM as a System Service	9
7.4 Modèles multimodaux on-device	10
7.5 Agents on-device	10
7.6 Benchmarks empiriques publiés (2024–2026)	10
8. Synthèse	11
Références	11

Chapitre 1 — État de l’Art : LLMs Embarqués sur Smartphone

Mémoire PFE — Intelligence Artificielle, Master Informatique
Rédigé en juillet 2026

1. Introduction et contexte

L’inférence de modèles de langage (LLM) directement sur des appareils mobiles représente l’une des évolutions les plus significatives du domaine de l’IA depuis 2023. Longtemps cantonnée aux serveurs cloud, l’exécution de LLMs sur smartphone est devenue techniquement viable grâce à la convergence de trois facteurs : la

miniaturisation des modèles (distillation, quantification), l'amélioration des SoCs mobiles (NPU/DSP), et le développement de frameworks d'inférence optimisés.

L'enjeu est considérable : il s'agit de permettre des interactions intelligentes en **temps réel, sans connexion réseau, avec préservation de la vie privée**, sur des appareils dont la RAM dépasse rarement 12 Go et dont la puissance de calcul représente une fraction d'un GPU de datacenter.

Ce chapitre dresse un panorama des solutions disponibles en 2025-2026, compare leurs performances, et analyse les différences architecturales entre exécution purement locale et architectures hybrides edge+cloud.

2. Panorama des modèles disponibles pour smartphone

2.1 Chronologie de l'émergence des LLMs mobiles

L'histoire des LLMs embarqués est récente mais dense :

Période	Événement clé
Août 2023	llama.cpp tourne sur iPhone (Georgi Gerganov) — démonstration de principe
Décembre 2023	Gemini Nano annoncé avec Pixel 8 Pro — premier LLM propriétaire on-device grand public
Février 2024	Google publie Gemma 2B/7B (open source, optimisé mobile)
Mars 2024	Microsoft publie Phi-3 Mini (3,8B, conçu pour mobile)
Juillet 2024	MLC-LLM 0.15 supporte les GPU Mali (MediaTek/Samsung)
Septembre 2024	Meta publie LLaMA 3.2 1B et 3B — premier Llama explicitement "mobile-first"
Octobre 2024	ML Kit GenAI (Google) disponible en preview — API Android pour Gemini Nano
Janvier 2025	Gemini Nano 2 déployé sur Pixel 9 et Galaxy S25
Avril 2025	Google publie Gemma 4 (variantes Edge 2B/4B, optimisées NPU)
Juin 2025	ML Kit GenAI v1.0 stable — API officielle pour développeurs Android

2.2 Familles de modèles et acteurs principaux

2.2.1 Google — Gemini Nano et Gemma **Gemini Nano** est le modèle propriétaire de Google destiné à l'exécution on-device. Il n'est pas distribué directement : les

développeurs y accèdent via l'API ML Kit GenAI ou AICore.

Version	Contexte	Disponibilité	Appareils
Gemini Nano 1	~2 048 tokens	Pixel 8 Pro uniquement	Historique (2023)
Gemini Nano 2	~2 048 tokens	Pixel 9/10, Galaxy S25/S26	Production (2025)
Gemini Nano 2 Multimodal	~2 048 tokens + vision	Pixel 9 Pro, Galaxy S25 Ultra	Production (2025)

Gemma est la famille open source de Google, disponible en GGUF pour llama.cpp ou via LiteRT/Google AI Edge.

Modèle	Paramètres	Taille Q4	Contexte	Benchmark (MMLU)
Gemma 2 2B IT	2,6 B	~1,5 Go	8 192	51,3 %
Gemma 2 9B IT	9 B	~5,5 Go	8 192	71,3 %
Gemma 3 4B IT	4 B	~2,5 Go	32 768	59,6 %
Gemma 4 E2B (Edge)	2 B	~1,2 Go	8 192	56 % (est.)
Gemma 4 E4B (Edge)	4 B	~2,4 Go	8 192	62 % (est.)

Les variantes **Edge** de Gemma 4 sont spécifiquement optimisées pour les NPU mobiles via le format LiteRT.

2.2.2 Meta — LLaMA 3.2 Meta a publié en septembre 2024 les premiers LLaMA explicitement conçus pour mobile :

Modèle	Paramètres	Taille Q4	Contexte	Benchmark (MMLU)
LLaMA 3.2 1B Instruct	1,24 B	~771 Mo	128 000	32,2 %
LLaMA 3.2 3B Instruct	3,21 B	~2,0 Go	128 000	58,0 %

La fenêtre de contexte théorique de 128 000 tokens est irréaliste en pratique mobile (RAM insuffisante). Les contextes pratiques sont de 2 048–4 096 tokens. LLaMA 3.2 1B est le modèle standard pour nos benchmarks : il constitue la référence minimale pour un déploiement ARM64.

2.2.3 Microsoft — Phi-3 / Phi-4 Mini Microsoft a adopté une approche “small but capable” avec la série Phi :

Modèle	Paramètres	Taille Q4	Contexte	Benchmark (MMLU)
Phi-3 Mini 4K	3,8 B	~2,3 Go	4 096	68,8 %
Phi-3 Mini 128K	3,8 B	~2,3 Go	128 000	68,8 %
Phi-3.5 Mini	3,8 B	~2,3 Go	128 000	69,0 %

Modèle	Paramètres	Taille Q4	Contexte	Benchmark (MMLU)
Phi-4 Mini	3,8 B	~2,3 Go	16 384	72,8 %

Phi-3/4 Mini se distingue par son score MMLU très élevé pour sa taille, au prix d'un fort biais anglophone et d'une performance dégradée en français.

2.2.4 Apple — OpenELM et Apple Intelligence Apple a adopté une approche radicalement fermée avec **Apple Intelligence** (iOS 18+), qui intègre plusieurs modèles on-device (2B–3B de paramètres estimés) via le Neural Engine. OpenELM (1B–3B) est la branche open source publiée pour la recherche.

Ces modèles sont inaccessibles sur Android et ne seront pas traités dans ce PFE.

2.2.5 Autres modèles notables

Modèle	Acteur	Paramètres	Particularité
Mistral 7B	Mistral AI	7,3 B	Trop lourd pour la majorité des smartphones (>4 Go Q4)
Qwen 2.5 1.5B	Alibaba	1,5 B	Excellent support CJK, bon support français
SmolLM 2 1.7B	Hugging Face	1,7 B	Conçu pour edge, contexte 8K
MobileLLM 1B	Meta Research	1 B	Recherche — non distribué publiquement
PowerInfer-2	SJTU	Variable	Exploite la sparsité des activations pour modèles 7B+

3. Comparaison des modèles : taille, latence, capacités

3.1 Tableau de comparaison général

Modèle	Params	Taille Q4	RAM min	Decode (tok/s)*	MMLU	Français	Licence
LLaMA 3.2 1B	1,2 B	771 Mo	3 Go	15–50	32 %	Moyen	Llama 3.2 Community

Modèle	Params	Taille Q4	RAM min	Decode (tok/s)*	MMLU	Français	Licence
LLaMA 3.2 3B	3,2 B	2,0 Go	5 Go	10-25	58 %	Bon	Llama 3.2 Community
Gemma 2 2B	2,6 B	1,5 Go	4 Go	12-35	51 %	Moyen	Gemma
Gemma 4 E2B	2 B	1,2 Go	3 Go	15-40	56 %	Bon	Gemma
Phi-4 Mini	3,8 B	2,3 Go	5 Go	8-20	73 %	Faible	MIT
Qwen 2.5 1.5B	1,5 B	900 Mo	3 Go	15-45	46 %	Très bon	Apache 2.0
Gemini Nano 2	~2 B (est.)	N/A (AICore)	Géré AICore	20-60**	~72 %	Bon	Propriétaire
SmolLM 2 1.7B	1,7 B	1,0 Go	3 Go	20-50	40 %	Moyen	Apache 2.0

* Decode en tokens/s sur un Snapdragon 8 Gen 2 ou équivalent, via llama.cpp (sauf mention).

** Estimation basée sur les benchmarks Google ; mesures exactes dépendantes de l'AICore.

3.2 Contraintes matérielles

L'exécution d'un LLM sur smartphone est soumise à des contraintes radicalement différentes du serveur :

RAM : La quantification est indispensable. Un modèle FP16 de 7B paramètres nécessite ~14 Go de RAM — hors de portée de tout smartphone actuel. Avec Q4_K_M, on descend à ~4 Go, viable sur appareils haut de gamme.

Contexte vs RAM : La fenêtre de contexte consomme de la RAM proportionnellement à sa taille (KV cache). En pratique : - 2 048 tokens : ~500 Mo supplémentaires pour un modèle 2B - 8 192 tokens : ~2 Go supplémentaires — critique sur 6 Go RAM

Thermique : Le throttling thermique est la principale source d'instabilité. Après 5-10 minutes d'inférence intensive, les SoCs réduisent leur fréquence de 20-40 %, dégradant les performances de façon notable (documenté dans le chapitre 2 de ce PFE).

NPU vs CPU : Les frameworks exploitant le NPU (AICore, LiteRT) peuvent offrir 2-4× le débit CPU à puissance équivalente, mais nécessitent la quantification INT8/INT4 et des formats propriétaires.

4. Frameworks d'inférence mobile

4.1 llama.cpp

Développé par Georgi Gerganov (2023), **llama.cpp** est devenu le standard de facto pour l'inférence de LLMs quantifiés sur CPU. Il utilise le format GGUF (GPT-Generated Unified Format).

Points forts : - Compatible avec tout ARM64 Android (pas de NPU requis) - Supporte Q2_K à Q8_0 — flexibilité de quantification - Très actif (commits quotidiens), large écosystème GGUF - Fonctionne via Termux sans root ni Android Studio

Points faibles : - CPU-only sur la grande majorité des appareils Android (pas de support GPU Mali) - N'exploite pas les NPU Snapdragon ni MediaTek - Performances limitées vs frameworks hardware-aware

Cas d'usage principal dans ce PFE : benchmark de référence sur CPU ARM64, tous appareils Android.

4.2 ML Kit GenAI (AICore)

Introduit par Google en 2024, **ML Kit GenAI** est l'API officielle pour accéder à Gemini Nano via l'AICore d'Android. L'AICore est un service système qui gère le modèle, son téléchargement, et l'accès au NPU.

Points forts : - Accès au NPU Snapdragon/Tensor — performances maximales - API simple (Kotlin/Java, quelques dizaines de lignes) - Gestion automatique du modèle (téléchargement, mise à jour) - Cas d'usage built-in : résumé, correction grammaticale

Points faibles : - Appareils certifiés seulement (Pixel 9/10, Galaxy S25/S26 en 2025) - Modèle non modifiable ou remplaçable - Exclut ~95 % du parc Android - Exige Android Studio pour le développement

Cas d'usage dans ce PFE : solution Google propriétaire, testée sur Galaxy S26 Ultra.

4.3 LiteRT (anciennement TensorFlow Lite)

LiteRT est le runtime d'inférence Google pour modèles .tflite. En 2024, Google a migré TFLite vers LiteRT et ajouté le support LLM via le framework "AI Edge LLM Inference".

Points forts : - Supporte NPU et GPU (délégés NNAPI, GPU, Hexagon) - Utilisé par Google AI Edge Gallery pour Gemma 4 Edge - Plus portable que ML Kit GenAI (pas de restriction AICore)

Points faibles : - Nécessite le format .task / .tflite (conversion depuis GGUF non triviale) - Écosystème moins mature que llama.cpp pour les modèles open source

4.4 MLC-LLM

MLC-LLM (Machine Learning Compilation for LLMs) du groupe MLC AI utilise Apache TVM pour compiler des modèles directement en code GPU/NPU optimisé.

Points forts : - Seul framework open source à exploiter les GPU Mali (Vulkan) et Adreno - Performances proches du NPU sur certains SoCs - Modèles pré-compilés disponibles pour Android

Points faibles : - Installation complexe (compilation requise pour chaque cible) - Taille des packages compilés importante - Moins de modèles disponibles que GGUF

4.5 MediaPipe LLM Inference

API Google (2024) intégrée dans MediaPipe, elle permet l'inférence de modèles Gemma au format .task :

MediaPipe → LiteRT runtime → CPU/GPU Adreno (Vulkan) / NPU

Points forts : - Support Vulkan (GPU Adreno) → exploite du hardware Snapdragon - Compatible avec Gemma 2/3/4, Phi-2, Falcon

Points faibles : - Limité aux modèles au format MediaPipe Task - Moins flexible que llama.cpp pour le choix du modèle

4.6 Comparaison des frameworks

Framework	CPU ARM64	GPU Mali	GPU Adreno	NPU Snapdragon	Facilité	Modèles
llama.cpp	☑ Excellent	☐ Non	☐ Non	☐ Non	☐☐☐☐	Tous GGUF
ML Kit GenAI	☐	☐	☐	☐ Natif	☐☐☐☐	Gemini Nano
LiteRT	☐	☐	☐	☐ NNAPI	☐☐☐	Gemma Edge
MLC-LLM	☐	☐ Vulkan	☐ Vulkan	☐	☐☐	Gemma, LLaMA
MediaPipe	☐	☐	☐ Vulkan	☐	☐☐☐	Gemma, Phi

5. Exécution locale pure vs architecture hybride (edge + cloud)

5.1 Exécution 100% locale

L'exécution entièrement locale signifie que le modèle tourne sur l'appareil, sans aucun appel réseau.

Avantages : - **Vie privée maximale** : les données ne quittent jamais l'appareil - **Disponibilité hors-ligne** : fonctionne sans connexion (zones rurales, avion, réseau instable) - **Latence déterministe** : pas de jitter réseau, temps de réponse prévisible - **Coût zéro** : pas d'abonnement API, pas de quota

Inconvénients : - **Qualité limitée** : les modèles <4B paramètres ne rivalisent pas avec GPT-4o ou Gemini Flash (cloud) - **Contexte court** : contrainte RAM → 2 048-4 096 tokens max en pratique - **Thermique** : sessions longues dégradées par le throttling - **Espace stockage** : modèles de 1-5 Go à télécharger

Cas d'usage idéaux : clavier prédictif, suggestions en temps réel, résumé de notes, extraction d'entités, chatbot FAQ offline.

5.2 Architecture hybride (edge + cloud)

L'architecture hybride combine un modèle local léger et un service cloud plus puissant, avec un routeur qui décide où exécuter chaque requête.

Requête utilisateur

↓

Routeur local
(règles + LLM léger)

↓

Tâche simple + contexte court
FAQ, résumé court, classification

→ Modèle local (Gemini Nano / Gemma 2B)
Réponse : 0,5–3 s

OU

Tâche complexe / contexte long
Raisonnement, analyse doc, code

→ API cloud (Gemini Flash / GPT-4o)
Réponse : 0,5–2 s (réseau)

Critères de routage typiques :

Signal	Action
Longueur du contexte > 1 500 tokens	→ Cloud
Question contenant des mathématiques / code complexe	→ Cloud
Appareil en mode offline	→ Local (forcé)
Requête de résumé < 500 mots	→ Local
Utilisateur hors données mobiles	→ Local

Avantages de l'hybride : - Meilleure qualité sur les tâches complexes - Économie de batterie (tâches simples restent locales) - Résilience offline partielle

Inconvénients : - Complexité architecturale accrue - Latence imprévisible (dépend du réseau pour les requêtes cloud) - Coût API pour la partie cloud

5.3 Positionnement des solutions étudiées

Solution	Mode	Cas d'usage principal
llama.cpp + LLaMA 3.2 1B	100% local	Benchmarking, apps offline, R&D
ML Kit GenAI (Gemini Nano)	100% local (AICore)	Apps grand public sur flagship
Google AI Edge Gallery	100% local (LiteRT)	Démonstration / évaluation Gemma Edge
Gemini Flash API	100% cloud	Référence qualité pour comparaison
Architecture hybride	Local + cloud	Production scalable

6. Critères de choix d'une solution

Le choix d'un framework et d'un modèle dépend de plusieurs critères interdépendants :

1. Parc cible : ML Kit GenAI est idéal si on cible uniquement les flagships 2024+ (Pixel 9/10, Galaxy S25/S26). Pour tout autre appareil Android, llama.cpp est la seule option CPU universelle.

2. Contrôle du modèle : Si le PFE ou l'application requiert un modèle fine-tuné ou personnalisé, ML Kit GenAI est exclu (modèle figé). llama.cpp ou MLC-LLM permettent de charger n'importe quel GGUF.

3. Performances : Sur les appareils compatibles, ML Kit GenAI via NPU surpasse llama.cpp (CPU). Sur les appareils non compatibles AICore, MLC-LLM via Vulkan offre les meilleures performances.

4. Vie privée : Les deux solutions (llama.cpp et ML Kit GenAI) sont locales. ML Kit GenAI peut potentiellement envoyer des métadonnées à Google (termes de service à vérifier).

5. Reproductibilité : llama.cpp est entièrement open source et reproductible sur n'importe quelle machine. ML Kit GenAI dépend d'un écosystème Google propriétaire susceptible d'évoluer.

7. Tendances et perspectives (2025-2027)

7.1 Compression agressive des modèles

La recherche sur la quantification INT2 (2 bits par paramètre) progresse rapidement. Des travaux comme QuaRot, GPTQ et BitNet.cpp suggèrent que des modèles 7B en INT2 (3,5 Go) deviendront viables sur les appareils haut de gamme 2026.

7.2 NPU comme cible principale

La tendance lourde est le passage du CPU au NPU comme cible d'inférence principale. Apple Neural Engine, Snapdragon NPU (Hexagon), et les NPU Google Tensor montrent des performances 5-10× supérieures au CPU à consommation équivalente.

7.3 LLM as a System Service

Des travaux académiques (Yin et al., 2024) proposent de déporter le modèle LLM au niveau du système d'exploitation, partagé entre applications — comme un service OS. Android 16 explore cette direction avec AICore. Cette approche éliminerait la duplication des modèles en mémoire (chaque app chargeant son propre modèle actuellement).

7.4 Modèles multimodaux on-device

Gemini Nano 2 Multimodal et Phi-3 Vision montrent que la vision (image → texte) devient accessible on-device en 2025. Les modèles audio (transcription) sont déjà matures (Whisper.cpp sur mobile).

7.5 Agents on-device

L'architecture agentique on-device (ReAct, Tool-calling) reste limitée par les capacités de raisonnement des petits modèles, mais progresse. Le protocole MCP (Anthropic, 2024) et les recherches sur les "Small Action Models" (SAM) ouvrent des perspectives pour 2026-2027.

7.6 Benchmarks empiriques publiés (2024-2026)

Plusieurs travaux récents mesurent les performances LLM sur appareils réels et constituent un cadre de comparaison direct pour les résultats de ce PFE.

PalmBench [22] (Li et al., 2024) propose un benchmark systématique de LLMs compressés sur plateformes mobiles (Google Pixel, iPhone) avec llama.cpp, couvrant latence CPU/GPU, consommation énergétique et empreinte mémoire. La méthodologie est la plus proche du protocole adopté dans ce PFE — nos mesures prefill/decode/throttling suivent une logique comparable, étendue aux appareils Samsung Exynos entrée/milieu de gamme absents de PalmBench.

MobileAIBench [23] (Murthy et al., NeurIPS 2024) évalue llama.cpp sur appareils réels avec 20 datasets couvrant NLP, tâches multimodales et sécurité. Son focus sur la diversité des évaluations qualitatives complète notre approche centrée sur les métriques d'exécution (tok/s, RAM, thermique).

Systematic Evaluation [24] (Song et al., 2025) évalue 7 méthodes PTQ sur des modèles de 0,5B à 14B paramètres et établit un seuil critique à 3,5 BPW : en dessous de cette densité de bits, la qualité chute de façon significative. Autre résultat notable : un grand modèle quantifié Q4 dépasse un petit modèle FP16 de taille inférieure. Le format Q4_K_M (~4,5 BPW) utilisé dans ce PFE est au-dessus du seuil critique — validation indépendante du choix de quantification.

LLM Inference at the Edge [25] (Tummalapalli et al., 2026) mesure un Galaxy S24 Ultra (Snapdragon 8 Gen 3) et un iPhone 16 Pro sous charge soutenue de 20 itérations avec Qwen 2.5 1.5B Q4. Résultat clé : le GPU du S24 Ultra subit un arrêt complet de l'inférence GPU par throttling thermique lors des sessions prolongées. Ce phénomène converge avec nos mesures sur Galaxy S26 Ultra (Snapdragon 8 Elite) : -17,3 % de dégradation thermique confirmée. Les deux appareils haut de gamme Snapdragon montrent une vulnérabilité thermique sous charge continue absente des appareils milieu de gamme.

PowerInfer-2 [26] (Xue et al., 2024) propose une décomposition en clusters de neurones atteignant 29,2× d'accélération, permettant théoriquement l'exécution d'un modèle 47B sur smartphone. Ces résultats, non encore reproduits de manière indépendante, représentent une direction de recherche avancée.

Positionnement du PFE : aucun de ces travaux ne couvre simultanément (1) les appareils Samsung Exynos entrée/milieu de gamme (A16, A26), (2) la comparaison systématique Termux vs UserLand, et (3) une mesure triple llama.cpp + LiteRT + cloud API sur le même appareil. Le PFE comble ces trois lacunes.

8. Synthèse

Axe	Situation 2025-2026	Horizon 2027
Modèles disponibles	1-4B paramètres viables	7-13B viables (INT2)
Frameworks matures	llama.cpp (CPU), ML Kit (NPU)	MLC-LLM + LiteRT convergence
Parc compatible NPU	Flagships uniquement (~5 %)	Haut/milieu de gamme
Qualité vs cloud	MMLU gap de 15-30 pts	Gap de 5-15 pts
Cas d'usage matures	FAQ, résumé, classification	Agents, RAG local
Thermique	Problème non résolu	Partielle amélioration (3nm+)

L'exécution de LLMs sur smartphone est passée en 2023-2025 du statut de curiosité technique à celui de réalité déployée sur des centaines de millions d'appareils (via Galaxy AI, Apple Intelligence). Les deux solutions étudiées dans ce PFE — llama.cpp côté open source, ML Kit GenAI / Gemini Nano côté Google — représentent les deux pôles de ce spectre : flexibilité universelle vs performance optimisée sur parc restreint.

Références

- Gerganov, G. (2023). *llama.cpp: Inference of Meta's LLaMA model in pure C/C++*. GitHub.
- Google DeepMind (2024). *Gemma: Open Models Based on Gemini Research and Technology*. arXiv:2403.08295.
- Meta AI (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783.
- Microsoft (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. arXiv:2404.14219.
- Xu et al. (2024). *Understanding LLMs Running on Consumer Devices*. arXiv:2410.03613.
- Yin et al. (2024). *LLM as a System Service on Mobile Devices*. arXiv:2403.11805.
- Google (2024). *ML Kit GenAI APIs*. developers.google.com/ml-kit/genai.
- Google (2024). *Android AICore*. developer.android.com/ml/aicore.
- Ye et al. (2025). *Prima.cpp: Speeding Up 70B-Scale LLM Inference on Low-Resource Everyday Home Clusters*. arXiv:2504.08791.

- Apple ML Research (2024). *Apple Intelligence Foundation Language Models*. arXiv:2507.13575.
- [22] Li et al. (2024). *PalmBench: A Comprehensive Benchmark of Compressed Large Language Models on Mobile Platforms*. arXiv:2410.05315.
- [23] Murthy et al. (2024). *MobileAIBench: Benchmarking LLMs and LMMs for On-Device Use Cases*. NeurIPS 2024. arXiv:2406.10290.
- [24] Song et al. (2025). *A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources*. arXiv:2505.15030.
- [25] Tummalapalli et al. (2026). *LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load*. arXiv:2603.23640.
- [26] Xue et al. (2024). *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. arXiv:2406.06282.
- [27] Yadav, M. & Bhargavi, P. (2024). *Optimizing LLMs Using Quantization For Mobile Execution*. ICT4SD 2025, Springer LNNS. arXiv:2512.06490.